Tema 1: Sistemul de fișiere
Varianta: 33968
Student: Mihai Adrian Diaconu

1 Descrierea temei

Se dă următorul format de fișier binar, pe care îl vom numi în continuare formatul **SF** (i.e. "**section file**").

Un SF este compus din două părți: **header** și **body**. Structura generală a unui SF este ilustrată mai jos. Se poate observa că **header**-ul este situat la începutul fișierului, înainte de **body**.

SF FILE STRUCTURE

```
HEADER
BODY
```

Header-ul unui SF conține informații care identifică formatul acestuia și deasemenea descrie modul în care body-ul trebuie citit. Aceste informații sunt organizate ca o secvență de câmpuri, fiecare câmp având un anumit număr de octeți și o anumită semnificație. Structura header-ului este ilustrată în caseta `HEADER` de mai jos, specificând pentru fiecare câmp numele acestuia și numărul de octeți pe care îl ocupă (separate prin ": "). Unele câmpuri sunt simple numere (ex. `MAGIC`, `HEADER_SIZE`, `VERSION` și `NR_OF_SECTIONS`), în timp ce altele (i.e. `SECTION_HEADERS` și `SECTION_HEADER`) au o structură mai complexă, având propriile sub-câmpuri. Astfel, `SECTION_HEADERS` este compus dintr-o secvență de elemente de tip `SECTION_HEADER`, fiecare dintre acestea având la rândul lui sub-câmpurile: `SECT_NAME`, `SECT_TYPE`, `SECT_OFFSET` și `SECT_SIZE`.

HEADER

```
MAGIC: 4
HEADER_SIZE: 2
VERSION: 2
NO_OF_SECTIONS: 1
SECTION_HEADERS: NO_OF_SECTIONS * sizeof(SECTION_HEADER)
    SECTION_HEADER: 11 + 2 + 4 + 4
        SECT_NAME: 11
        SECT_TYPE: 2
        SECT_OFFSET: 4
        SECT_SIZE: 4
```

Semnificația fiecărui câmp este următoarea:

- Câmpul `MAGIC` identifică fișierele SF. Valoarea acestuia trebuie să fie `"BTT1"`.

- Câmpul `HEADER_SIZE` indică dimensiunea header-ului fișierului SF.
- Câmpul `VERSION` identifică versiunea formatului SF, presupunând că acesta se poate schimba de la o versiune la alta (deși nu este cazul la această temă).
- Câmpul `NO_OF_SECTIONS` specifică numărul de elemente de tip `SECTION_HEADER`, care vor fi acoperite mai jos.
- Sub-câmpurile unui `SECTION_HEADER` fie au nume explicite, fie sunt descrise mai jos.

Body-ul unui SF, practic o secvență de octeți, este organizat ca o colecție de secțiuni. O secțiune e formată dintr-o secvență de `SECT_SIZE` octeți consecutivi, începând de la octetul `SECT_OFFSET`, în cadrul fișierului. Secțiunile consecutive nu vor fi neapărat una după alta. Cu alte cuvinte, între două secțiuni consecutive pot exista octeți ce nu aparțin nici unei secțiuni. O secțiune dintr-un SF conține caractere afișabile și caractere de final de linie. Se poate spune că sunt de fapt secțiuni de tip text, formate din mai multe linii. Octeți dintre secțiuni pot avea orice valoare, dar aceștia nu au nici o relevanță în interpretarea conținutului unui SF.

În cadrul unei secțiuni, numerotarea liniilor se face de la început, începând cu indicele 1.

Următorele restricții se aplică anumitor câmpuri din SF:

- Valoarea câmpului `MAGIC` este `"BTT1"`.
- Valoarea câmpului `VERSION` e un număr între 68 și 155, inclusiv.
- Câmpul `NO_OF_SECTIONS` are valoarea 2 sau o valoare între 4 și 16, inclusiv.
- Valorile valide pentru `SECT_TYPE` sunt: 70 64 77 93 (în baza 10).
- Liniile dintr-o secțiune sunt separate de următoarea secvență de octeți (valorile sunt în hexazecimal): `0A`.

Figura 1 ilustrează un exemplu de fișier SF format din următoarele elemente:

- `HEADER`: de la offset-ul 0x00 până la offset-ul 0x32 (adică la începutul fișierului)
  - `MAGIC`: la offset-ul 0x00, cu valoarea `"BTT1"`
  - `HEADER_SIZE`: la offset-ul 0x04, cu valoarea 51 (0x33)
  - `VERSION`: la offset-ul 0x06, cu valoarea 71 (0x47)
  - `NO_OF_SECTIONS`: la offset-ul 0x08, cu valoarea 2
  - `SECTION_HEADERS`: de la offset-ul 0x09 până la offset-ul 0x32:
    - $\texttt{SECTION_HEADER}_1$
      - $\texttt{SECTION_NAME}_1$: la offset-ul 0x09, cu valoarea “sect1”
      - $\texttt{SECTION_TYPE}_1$: la offset-ul 0x14, cu valoarea 70 (0x46)

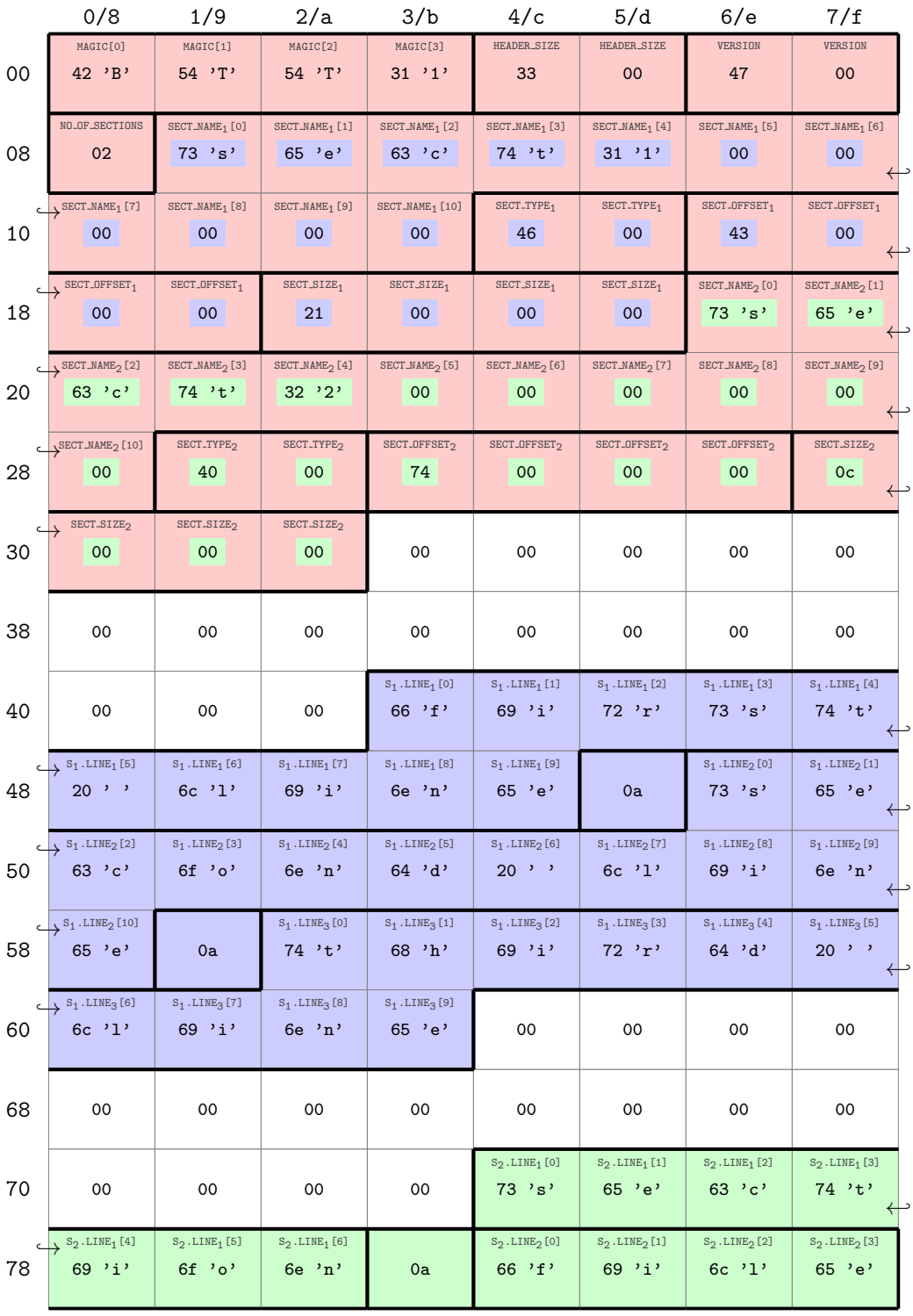


Figure 1: Exemplu de fișier SF

        - SECTION_OFFSET$_1$: la offset-ul 0x16, cu valoarea 0x43; acest lucru înseamnă că secțiunea 1 începe în fișier la adresa 0x43
        - SECTION_SIZE$_1$: la offset-ul 0x1a, cu valoarea 33 (0x21); acest lucru înseamnă că secțiunea 1 ocupă în fișier 33 octeți
    * SECTION_HEADER$_2$
        - SECTION_NAME$_2$: la offset-ul 0x1e, cu valoarea "sect2"
        - SECTION_TYPE$_2$: la offset-ul 0x29, cu valoarea 64 (0x40)
        - SECTION_OFFSET$_2$: la offset-ul 0x2b, cu valoarea 0x74; acest lucru înseamnă că secțiunea 2 începe în fișier la adresa 0x74
        - SECTION_SIZE$_2$: la offset-ul 0x2f, cu valoarea 12 (0xc); acest lucru înseamnă că secțiunea 2 ocupă în fișier 12 octeți
- secțiunea 1 : de la offset-ul 0x43 până la offset-ul 0x63
    - linia 1 , de la offset-ul 0x43 până la offset-ul 0x4c cu conținutul "first line"
    - linia 2 , de la offset-ul 0x4e până la offset-ul 0x58 cu conținutul "second line"
    - linia 3 , de la offset-ul 0x5a până la offset-ul 0x63 cu conținutul "third line"
- secțiunea 2 : de la offset-ul 0x74 până la offset-ul 0x7f
    - linia 1 , de la offset-ul 0x74 până la offset-ul 0x7a cu conținutul "section"
    - linia 2 , de la offset-ul 0x7c până la offset-ul 0x7f cu conținutul "file"

2 Cerințele temei

Trebuie să scrieți un program C, numit "*a1.c*" care implementează cerințele ce urmează.

2.1 Compilarea și rularea

Programul trebuie să poată fi **compilat fără erori**, pentru a fi acceptat. Comanda de compilare va fi cea de mai jos:

Compilation Command

```
gcc -Wall a1.c -o a1
```

Warning-uri în procesul de compilare vor aduce o penalizare de 10% din scorul final.

Atunci când rulează, executabilul vostru (îl vom numi "*a1*") **trebuie să ofere funcționalitatea minimală și rezultatele așteptate**, pentru a fi acceptat. Vom defini mai jos ce înseamnă funcționalitatea minimală. Următoarea casetă ilustrează modul în care programul va fi rulat. Opțiunile și parametrii pe care programul trebuie să îi accepte, împreună cu semnificația acestora se vor detalia în următoarele secțiuni.

Running Command

```
./a1 [OPTIONS] [PARAMETERS]
```

2.2 Afișarea variantei

Fiecare student primește o variantă puțin modificată a formatului SF și a cerințelor. Prima sarcină din temă va fi afișarea variantei de temă primită, atunci când programul este rulat precum mai jos.

Display Variant Command

```
./a1 variant
```

Output Sample for Display Variant Command

```
33968
```

2.3 Afișarea conținutului folderelor

Atunci când se dă opțiunea "`list`", programul trebuie să afișeze pe ecran numele anumitor elemente (ex. fișiere, subfoldere) din folderul a cărui cale este specificată prin opțiunea "`path`", conform exemplului din caseta de mai jos.

List Directory Command

```
./a1 list [recursive] <filtering_options> path=<dir_path>
```

Opțiunile din linia de comandă se pot da în orice ordine, de exemplu opțiunea "recursive" poate apărea înainte sau după alte opțiuni.

Elementele care trebuie afișate vor fi determinate pe baza criteriilor de filtrare menționate în continuare. Fiecare nume de element trebuie afișat pe o linie separată, fără linii libere între nume, în forma ilustrată în caseta de mai jos (numele fiecărui element trebuie să înceapă cu calea acestuia, specificată prin opțiunea "path"). Dacă nu se găsește nici un element care să corespundă criteriilor de căutare, nu trebuie afișat decât string-ul "SUCCESS".

Sample Output for List Directory Command (Success Case)

```
SUCCESS
test_root/test_dir/file_name_1
test_root/test_dir/file_name_2
test_root/test_dir/file_name_3
...
```

Dacă se folosește opțiunea "`recursive`", programul vostru trebuie să traverseze întregul sub-arbore, începând din orice folder dat, intrând recursiv în toate sub-folderele. Elementele găsite trebuie de asemenea să conțină calea cu tot cu directorul specificat prin opțiunea "path". Cele două casete de mai jos arată cum se poate rula programul cu opțiunea "`recursive`" și un output posibil.

Sample List Directory Command

```
./a1 list recursive path=test_root/test_dir/
```

Sample Output for List Directory Command (Success Case)

```
SUCCESS
test_root/test_dir/file_name_1
test_root/test_dir/file_name_2
test_root/test_dir/subdir_1/file_name_1
test_root/test_dir/subdir_1/subdir_1_1/file_name_2
test_root/test_dir/subdir_2/file_name_3
...
```

În cazul în care se întâlnește o eroare, trebuie afișat un mesaj de eorare, urmat de o explicație, conform casetei următoare.

Output for List Directory Command (Error Case)

```
ERROR
invalid directory path
```

Criteriile de filtrare, utilizate pentru a selecta elementele ce vor fi afișate sunt următoarele:

- Numele elementului trebuie să înceapă cu secvența de caractere specificată prin opțiunea "`name_starts_with`=string". Trebuie considerate toate tipurile de elemente, atât fișirele cât și folderele.
- Elementele afișate trebuie să aibă permisiunea de execuție pentru proprietar (eng. *owner*), dacă se specifică opțiunea "`has_perm_execute`". Trebuie considerate toate tipurile de elemente, atât fișirele cât și folderele.

Deși în practică se pot folosi mai multe criterii de filtrare în aceeași comandă, testele noastre folosesc un singur criteriu la un moment dat. Programul vostru ar trebui să trateze cazul general, dar aceasta nu e o cerință obligatorie.

Ordinea numelor de elemente afișate nu este importantă în aceste teste, doar numărul și valoarea acestora.

2.4 Identificarea și parsarea fișierelor SF

Atunci când se folosește opțiunea "`parse`", programul vostru trebuie să verifice dacă fișierul a cărui cale a fost specificată prin opțiunea "`path`" respectă sau nu formatul SF. În următoarea casetă se ilustrează modul de rulare a programului.

Check SF Format Command

```
./a1 parse path=<file_path>
```

Argumentele se pot da în orice ordine.

Validarea formatului SF se face după următoarele reguli:

- Valoarea câmpului MAGIC este cea menționată mai sus, adică "BTT1".
- Valoarea câmpului VERSION e un număr din intervalul menționat mai sus, adică între 68 și 155, inclusiv.
- Numărul de secțiuni trebuie să fie 2, sau o valoare între 4 și 16, inclusiv.

- Tipurile secțiunilor din fișier trebui să fie dintre următoarele valori: 70 64 77 93 .

Atunci când toate criteriile de mai sus sunt îndeplinite, fișierul se poate considera valid, chiar dacă există alte inconsistențe (exemple: dimensiunea fișierului e prea mică sau secțiunile se suprapun).

În caz că fișierul dat nu este valid, un mesaj de eroare trebuie afișat pe ecran, urmat de primul motiv pentru care fișierul este invalid, conform casetei de mai jos.

Sample Output for an Invalid SF File

```
ERROR
wrong magic|version|sect_nr|sect_types
```

În caz că fișierul dat este valid (respectă formatul SF), anumite câmpuri trebuie afișate pe ecran, în forma ilustrată.

Sample Output for a Valid SF File

```
SUCCESS
version=<version_number>
nr_sections=<no_of_sections>
section1: <NAME_1> <TYPE_1> <SIZE_1>
section2: <NAME_2> <TYPE_2> <SIZE_2>
...
```

2.5 Lucrul cu secțiuni

Atunci când se folosește opțiunea "`extract`", programul vostru trebuie să caute și să afișeze o anumită porțiune a fișierului SF. În mod particular, o singură linie trebuie să fie extrasă. Argumentele din linia de comandă necesare sunt "`path`", "`section`" și "`line`", specificând calea spre fișier, numărul secțiunii, respectiv numărul liniei, conform casetei de mai jos.

Extract Section Line Command

```
./a1 extract path=<file_path> section=<sect_nr> line=<line_nr>
```

În cazul în care apare o eroare, mesajul de eroare trebuie afișat, urmat de motivul erorii, ca și în caseta de mai jos.

Sample Output for Extract Section Line Command (Error Case)

```
ERROR
invalid file|section|line
```

În cazul în care fișierul dat respectă formatul SF iar secțiunea și linia căutată există, rezultatul trebuie afișat precum în caseta de mai jos.

Sample Output for Extract Section Line Command (Success Case)

```
SUCCESS
<line_content>
```

Liniile se numără de la începutul secțiunii, prima linie având numărul 1.

2.6 Filtrarea după secțiuni

Atunci când se dă opțiunea "`findall`", programul vostru trebuie să funcționeze ca și în cazul opțiunilor "`list recursive`", dar se vor afișa doar fișierele SF valide, care

au cel puțin 2 secțiuni de tipul 93.

Modul de rulare a programului este ilustrat mai jos:

Find Certain SF Files Command

```
./a1 findall path=<dir_path>
```

În caz că se întâmpină erori, se va afișa un mesaj de eroare, urmat de o explicație, în forma de mai jos:

Output for Find Certain SF Files Command (Error Case)

```
ERROR
invalid directory path
```

Output-ul normal va începe cu linia "SUCCESS", urmată de fișierele găsite, câte unul pe linie:

Sample Output for Find Certain SF Files Command (Success Case)

```
SUCCESS
test_root/test_dir/file_name_1
test_root/test_dir/file_name_2
...
```

3 Manual de utilizare

3.1 Auto-evaluare

Pentru a genera și rula testele pentru soluția voastră, trebuie rulat script-ul "tester.py", furnizat împreună cu cerințele. Script-ul are nevoie de Python 3.x, nu de Python 2.x.

Sample Command for Tests Generation

```
python3 tester.py
```

Chiar dacă script-ul funcționează și pe Windows (și pe alte sisteme de operare), recomandăm rularea pe Linux, deoarece așa se va evalua tema voastră.

Atunci când se rulează script-ul, se va genera un folder numit "test_root", ce conține diverse sub-foldere și fișiere, pe care se va testa soluția voastră. Chiar dacă conținutul lui "test_root" este aleator și diferit pentru fiecare student, generarea acestuia se face în mod determinist, indiferent de timpul rulării script-ului. Pot totuși apărea diferențe dacă se rulează pe alte sisteme de operare.

După crearea folderului "test_root" și a conținutului acestuia, script-ul va rula de asemenea programul vostru, afișând rezultatul diferitelor teste. În mod normal vor fi 65 de teste, deși pot fi și mai puține. Nota maximă (10) se obține atunci când trec toate testele, fără nici o penalizare (vezi mai jos). Nota exactă se calculează scalând numărul de teste trecute în intervalul 0-10.

Restricții:

- Sunteți limitați la a folosi doar apeluri de sistem către sistemul de operare (funcții low-level). De exemplu, TREBUIE folosi funcțiile `open()`, `read()`, `write()` etc., nu funcțiile de nivel înalt `fopen()`, `fgets()`, `fscanf()`, `fprintf()` etc. Singurele excepții sunt citirea de la `STDIN` și afișarea în `STDOUT` / `STDERR`, cu funcții precum `scanf()`, `printf()`, `perror()`, respectiv funcții de manipulare a string-urilor cum ar fi `sscanf()`, `snprintf()`.

Recomandări:

- Pentru tokenizarea string-urilor (separarea după elemente specifice, cum ar fi spații) recomandăm utilizarea funcțiilor `strtok()` sau `strtok_r()`.

Dacă doriți să vedeți detalii legate de testele eșuate, puteți folosi opțiunea `--verbose`, care va afișa atât rezultatul așteptat de test cât și rezultatul programului vostru. Cu opțiunea `--test test_name` puteți rula un singur test, cu numele dat, în vederea depanării. De exemplu dacă doriți să rulați doat testul numit `parse_2` și să vedeți detaliile, puteți combina cele două opțiuni, precum mai jos:

Sample command for single test with verbose
```
python3 tester.py --test parse_2 --verbose
```

Atunci când tema voastră este evaluată pentru notă, aceasta se rulează într-un container de *docker*. Chiar dacă o soluție corectă și deterministă se va comporta la fel, un bug în soluție poate trece neobservat pe sistemul vostru dar poate produce un crash / comportament nedefinit la evaluarea "oficială". Din acest motiv vă încurajăm să vă testați soluția și cu docker înainte să o trimiteți. Pentru asta trebuie să:

- instalați *docker* pe sistem (`sudo apt install docker.io`)
- creați un cont pe Docker Hub și să vă logați din linia de comandă (e necesar pentru a descărca imaginea de evaluare)
- instalați modulul *docker* pentru Python (`python3 -m pip install docker`)

Ca să vă testați soluția folosind un container de *docker* trebuie să dați argumentul `--docker` script-ului de testare.

Sample command for testing with Docker
```
python3 tester.py --docker
```

Dacă doriți să păstrați container-ul de docker ulterior rulării testelor (de exemplu pentru a putea face debug în interiorul acestuia), folosiți opțiunea `--docker-persist`.

3.2 Evaluare și cerințe minime

- Dacă apar warning-uri la compilare, se va aplica o penalizare de 10%.

- Dacă programul vostru are memory leak-uri (memorie alocată și ne-eliberată) găsite de *valgrind*, se va aplica o penalizare de 10%.
- Dacă programul nu respectă stilul de cod, se poate aplica o penalizare de până la 10% (decisă de cadrul didactic de la laborator).

Nu încercați să trișați, deoarece se vor rula tool-uri de detecție a temelor plagiate.

Notă: Testele furnizate nu sunt neapărat aceleași cu cele pe care se va testa soluția voastră. Nu încercați să afișați rezultatele așteptate pe baza numelor fișierelor. Deasemenea, codul vostru trebuie să îndeplinească toate cerințele, chiar dacă unele dintre ele nu sunt acoperite de teste. Este posibil ca unele cazuri mai rare să nu apară în testele voastre, dar să fie testate la evaluare.